The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5 | Max Marks: 100

Student Name	Registration Number
BANWARI MEWADA	24MIM10236
Slot	**Date of Submission**
B22	31-03-2026

Chosen Software: Git (Version Control · GPL v2)

"The tool Linus built when proprietary failed him"

1. Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers rely on daily, the operating systems inside supercomputers and smartphones — wasn't built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That's the essence of open source. And honestly, when you stop and think about it, that's remarkable.

This project is an invitation to stop and think about exactly that.

I've chosen Git as the subject of this audit — specifically Version 2.x, licensed under GPL v2. The choice felt obvious once I started researching. Git isn't just another utility. It's the invisible backbone that most modern software development rests on. Every commit you push, every branch you merge, every pull request you open — Git is underneath all of it. What I find interesting is that it was built over a single weekend by one person who was annoyed enough to do something about a broken system.

This report covers the origin story of Git, the freedoms its license actually grants (and what that means in practice), the ethical questions open source raises, how Git lives on a Linux system, the ecosystem it has spawned, and an honest comparison against proprietary alternatives. Alongside that, I've written five shell scripts that demonstrate practical Linux skills — not impressive programs, just evidence that I understand how these tools work at the command line.

Why this matters *Every line of code you'll write in your career will sit on top of open-source foundations. Understanding the philosophy behind that isn't optional — it's part of being a professional.*

2. Choosing Your Software

From the list of approved software options, I have selected:

Software	Category	License	Why it's interesting
Git	Version Control	GPL v2	The tool Linus built when proprietary failed him

3. Part A — Origin and Philosophy

A1: The Problem Git Was Created to Solve

The World Before Git (Pre-2005)

To understand why Git exists, you have to understand what version control looked like before 2005 — and more specifically, what the Linux kernel development team was dealing with. It's not a pretty picture.

The Linux kernel is one of the most complex collaborative software projects in human history. By the early 2000s, thousands of developers scattered across the globe were contributing patches, fixes, and features. Managing all of that with a coherent, non-destructive workflow is genuinely hard. The tools that existed at the time were either frustrating, slow, or philosophically incompatible with how open-source development actually works.

CVS — the Concurrent Versions System — was the dominant tool for many years. And it was, frankly, a mess. Atomic commits didn't exist in CVS: if your commit failed halfway through, you'd end up with a broken, partially-applied state. Branching was so painful that most developers avoided it entirely. Renaming a file was treated as deleting the old one and creating a new one, losing all history in the process. It was functional in the way that a bicycle with square wheels is functional — it technically moves, but nobody enjoys riding it.

SVN (Subversion) came along to fix some of these problems and it did, partially. Atomic commits arrived. Directory versioning improved. But SVN was still fundamentally centralized: there was one server, one repository, and if you lost network connectivity — or worse, if that central server went down — your workflow stopped dead. For the globally distributed, often-offline nature of kernel development, that was a serious problem.

So what did the Linux kernel team actually use? For a period starting in 2002, they used BitKeeper — a proprietary distributed version control system developed by a company called BitMover. It was genuinely good software and it was offered to the kernel team for free (as in price, not freedom). This arrangement worked until 2005, when BitMover decided the free access was over.

The BitKeeper Breakdown

The relationship between the Linux kernel project and BitMover soured over an incident involving a developer (Andrew Tridgell) who had been reverse-engineering the BitKeeper protocol — trying to understand how it worked and potentially create a compatible free alternative. BitMover saw this as a violation of the spirit of the arrangement and revoked the free license.

Whether this was a fair call or not is beside the point. What mattered was the consequence: thousands of kernel developers suddenly didn't have a version control system. And not just any project — the Linux kernel, arguably the most important open-source project on the planet.

Linus Torvalds evaluated the alternatives. CVS and SVN were out — too slow, too centralized, fundamentally not designed for the scale of work the kernel required. He considered other distributed systems like Monotone, but found them too slow for his needs. So he did what engineers sometimes do when nothing on the market is good enough: he decided to build something himself.

A1: The Solution — Git's Creation

Why Linus Built It Himself

This is the part of the origin story I find genuinely interesting. Linus didn't set out to build a version control system. He set out to solve a very specific problem: how do you manage a repository the size of the Linux kernel, with patches coming in from thousands of contributors, in a way that's fast, reliable, and distributed?

He started working on Git in April 2005 and had the first version managing its own development in a matter of days. The initial self-hosted commit happened on April 7, 2005. By June, Linus had handed off maintenance to Junio Hamano, who still maintains the project today. That's an extraordinary timeline. Most software projects spend months just arguing about architecture.

The Core Design Philosophy

Git was built around a set of design goals that were direct responses to the failures of previous tools. And because Linus designed it himself, with his own extremely demanding use case in mind, those goals are quite specific:

- Distributed by default. There is no single central server in Git's model. Every developer has a complete copy of the entire repository, including full history. This means you can commit, branch, and merge entirely offline. When you're ready to share, you push. This was non-negotiable for a project like the Linux kernel.
- Speed above almost everything else. Linus explicitly set a performance requirement: operations that take seconds in other systems should take milliseconds. He benchmarked against CVS and used it as a lower bound. This is why Git's internal data structures are designed the way they are — everything is optimized for reading and writing large amounts of data quickly.
- Strong data integrity. Every object in Git — every commit, every file snapshot, every tree — is identified by its SHA-1 hash. This isn't just an identifier: it's a cryptographic checksum of the content itself. You literally cannot change a historical commit without changing its hash, and therefore breaking every reference to it. This gives Git an almost automatic audit trail.
- Support for non-linear development. Branching and merging had to be cheap and fast. The Linux kernel has hundreds of active development branches at any given time. Git's model makes creating a branch a nearly instantaneous operation — it's just a file containing a hash. Merging, while sometimes complex, is a first-class operation in Git.

What I find notable is that these weren't features Linus added later — they were the requirements he started with. He knew exactly what he needed because he'd been living with tools that couldn't deliver it.

A2: The License — What GPL v2 Actually Says

The Four Freedoms of Free Software

The philosophical framework behind Git's license comes from the Free Software Foundation, founded by Richard Stallman in 1985. The FSF defined four fundamental freedoms that a software license must grant for the software to be considered 'free' in the meaningful sense of the word. Git's GPL v2 license grants all four:

- Freedom 0 — The freedom to run the program as you wish, for any purpose. You can use Git for your personal projects, your company's infrastructure, or building a commercial product on top of it. There are no restrictions on use.
- Freedom 1 — The freedom to study how the program works and change it. This requires access to the source code, which GPL v2 mandates. You can read every line of Git's code, understand it, and modify it to suit your needs.
- Freedom 2 — The freedom to redistribute copies. You can share Git with anyone. You can burn it to a USB drive and give it to a friend who's never heard of it. You can package it in a Linux distribution and ship it to millions of users.
- Freedom 3 — The freedom to distribute copies of your modified versions. If you improve Git, you can share those improvements. This is the freedom that makes collaborative open-source development actually work.

All four freedoms are present in Git's GPL v2 license. But GPL v2 adds a critical condition that transforms these freedoms from passive permissions into an active requirement — and that's where copyleft comes in.

Copyleft: Can a Company Modify Git and Sell It Without Sharing?

No. That's the short answer, and it's the defining characteristic of copyleft licensing.

Under GPL v2, if you distribute a modified version of Git — as part of a product, as a service, in any form — you must also distribute the complete corresponding source code of your modifications, under the same GPL v2 license. This is what 'copyleft' means: the license uses copyright law to ensure freedom propagates forward, not back.

So if a company takes Git, adds proprietary features to it, and sells that modified version, they are legally required to release their modifications under GPL v2. They can charge for it — the license doesn't prohibit that — but they can't keep the improvements secret. The source has to be available.

There's an important nuance here: if you use Git as a tool internally without distributing it, GPL v2 imposes no obligation. A company running Git on its own servers for its own developers doesn't have to release anything. The copyleft clause is triggered by distribution, not use.

A2: GPL vs MIT — A Contrast in Philosophy

The Fundamental Difference

MIT is what's called a permissive license. It grants essentially unlimited freedom with one condition: keep the copyright notice and license text intact. That's it. You can take MIT-licensed code, incorporate it into a proprietary product, never share your changes, and sell the whole thing. The MIT license doesn't care.

GPL, by contrast, is a 'share-alike' or copyleft license. It grants the same freedoms, but adds the condition that those freedoms must be preserved when you distribute. It's a philosophical difference that runs deep. MIT says 'do whatever you want.' GPL says 'do whatever you want, but if you pass it on, pass the freedom along too.'

Aspect	GPL v2	MIT
Modification allowed?	Yes	Yes
Commercial use?	Yes, but source must be shared	Yes, source can remain private
Must share changes?	Yes (if distributing)	No
Copyleft?	Strict	None
Use in proprietary products?	Only if GPL-compliant	Yes, freely

Which Would I Choose?

For a personal project I genuinely want people to use and build on — something like a library or a utility — I'd probably lean toward MIT. It removes friction. Developers at companies with strict legal departments often can't use GPL-licensed code at all because the copyleft clause creates legal uncertainty. If I want maximum adoption, MIT wins.

But if I were building something I believed in philosophically — something where I didn't want large corporations to take the work, privatize it, and never contribute back — I'd use GPL. Git is a good example of why that choice makes sense. The fact that companies can't simply absorb Git and make it proprietary is exactly why Git has remained community-owned and freely available for twenty years.

'Free as in Beer' vs 'Free as in Freedom'

This is one of those distinctions that sounds trivial but actually matters a lot. Richard Stallman formalized it, but the idea is straightforward. 'Free as in beer' means price: you get something for nothing. 'Free as in freedom' means liberty: you have the right to use, understand, modify, and share it.

Git is both. You can download it for free (no cost), and you have all four freedoms to use, study, modify, and redistribute it. But it's the second kind of free that matters. Imagine if Git were free to download but closed-source. You couldn't read the code to understand a bug. You couldn't modify it to support your specific workflow. You'd be entirely dependent on the maintainers to fix problems and add features. That's the difference between a tool you own and a tool you're borrowing.

A3: The Ethics of Open Source

Should All Software Be Open Source?

The Case For

There's a compelling argument that software, as a form of knowledge, should be freely shared — the same way we don't copyright mathematical proofs or scientific methods. Software increasingly governs critical infrastructure: hospitals, power grids, financial systems, election machinery. When the code running these systems is opaque, there's no way for the public to verify it's working correctly, or to detect when it isn't. Transparency isn't just a nice-to-have; it's arguably a public safety issue.

There's also the innovation argument. Open source has produced Git, Linux, Python, PostgreSQL, Firefox — tools that have collectively enabled trillions of dollars of economic value and accelerated human knowledge in ways that proprietary alternatives simply couldn't have, because proprietary software optimizes for profit, not for collective benefit.

The Case Against

The counterargument is economic reality. Building good software is expensive. It requires time, expertise, and sustained attention. Companies invest in software development because they expect a return. If you mandate open source, you potentially remove the financial incentive that funds large-scale software development. Who pays the engineers who maintain complex, critical software? Who funds the security audits?

There are also legitimate cases for private software: medical devices with proprietary algorithms that represent genuine competitive investment, security software where revealing the source code helps attackers, niche enterprise tools where the small user base makes open-source community maintenance impractical.

My own view? The argument for openness is strongest for infrastructure-level software — things the entire ecosystem depends on. It's weakest for narrow, specialized applications. The line isn't always obvious.

Is It Ethical to Profit From Free Community Labor?

This is a genuinely difficult question and I want to think through it carefully rather than just assert an answer.

Red Hat built a billion-dollar company (now part of IBM) on top of Linux and other open-source software. Google runs on Linux. AWS's compute infrastructure is Linux. Meta has contributed back to the open-source ecosystem — but it has also taken enormously from it. Is that ethical?

I think the key distinction is between participation and extraction. Red Hat contributed back extensively — through code, through employing kernel developers, through funding community infrastructure. That feels different from a company that takes open-source software, wraps it in a proprietary interface, and never contributes anything upstream. The latter feels like a violation of the spirit of open source, even if it's technically legal under permissive licenses.

This is actually one of the arguments for GPL over MIT: GPL creates a structural incentive to contribute back, because the alternative — keeping your changes private while distributing the software — is prohibited.

'Standing on the Shoulders of Giants'

Newton's phrase, adapted for software: every piece of code anyone writes today exists because of decades of prior work by people who chose to build in the open. The programming languages, the compilers, the operating systems, the networking stacks, the databases — almost all of it open source, almost all of it free.

Git itself stands on shoulders. It builds on zlib for compression, libcurl for network transfers, OpenSSL for cryptography, POSIX for portability. None of those were built by Linus. All of them were freely available because their authors chose to share them.

Does open source undermine original innovation? I don't think so. What it does is shift the location of competitive advantage — away from hoarding knowledge and toward applying it. The companies that do best in an open-source world are the ones who are best at using and extending common foundations, not the ones who lock things down. That seems like a healthier equilibrium.

4. Part B — Linux Footprint

Installation

Installing Git on a Debian/Ubuntu-based system (including WSL) is straightforward via the apt package manager. Running the following command pulls Git from the distribution's official package repositories, verifies the package signature, and handles all dependencies automatically:

```
sudo apt update
sudo apt install git
```

This is fundamentally different from installing proprietary software, where you download a binary and trust that it does what it says. With apt, you're pulling from a signed repository — the distribution maintainers have audited and packaged the software, and the signature chain gives you cryptographic assurance that it hasn't been tampered with. The transparency runs all the way down.

After installation, confirming the version with git --version shows something like git version 2.43.0. The exact version depends on which release your distribution has packaged — Ubuntu tends to be conservative and slightly behind the latest upstream release.

Key Directories and Files

Git's footprint on a Linux system is clean and well-organized, following standard POSIX filesystem conventions:

- /usr/bin/git — the main Git executable. This is where the binary lives after installation via apt. When you type git at the command line, this is what runs.
- /usr/lib/git-core/ — a directory full of sub-commands. Git's architecture is modular: git commit, git merge, and git push are implemented as separate executables (git-commit, git-merge, git-push) that the main git binary dispatches to. This is Unix philosophy in action — small tools doing one thing well.
- ~/.gitconfig — the per-user configuration file. This is where your name, email, preferred editor, aliases, and other personal settings live. It's a plain-text INI-format file, readable and editable directly.
- /etc/gitconfig — the system-wide configuration. Settings here apply to all users on the machine. User-level settings in ~/.gitconfig override these where they conflict.
- ~/.git/ (inside any repository) — the repository's local data. This hidden directory contains the full history, all branches, all commit objects, and Git's internal databases. The entire version control state of a project lives here.

User, Group, and Security

Git doesn't run as a privileged user. When you execute git commands, they run as your own user account with your own file permissions. This is correct behavior — Git has no reason to touch files you don't own, and elevating its privileges would be a security risk.

The security implication of .git/ directories deserves mention. The objects stored there include every version of every file you've ever committed, including files you've since deleted. Accidentally committing secrets (API keys, passwords) means those secrets persist in history even if you remove the file — which is why tools like git-secrets and .gitignore exist, and why understanding what Git stores is practically important.

For server-side Git (bare repositories shared among a team), the user/group model becomes important. A shared repository owned by a git user with permissions granted to a git group lets multiple people push to the same repository without exposing each other's files. Git hosting platforms like GitHub handle all of this infrastructure on your behalf, but understanding the underlying model explains why those platforms work the way they do.

Service Management

Git is not a persistent background daemon. Unlike Apache or MySQL, which run continuously and listen for connections, Git is entirely command-driven: it runs when you call it and exits when the command completes. There's no git start or git stop. There's nothing to restart.

This is actually consistent with Git's distributed philosophy. The 'server' in a Git workflow is simply a machine with a repository that others have push access to. If you want a networked Git server, you run git-daemon (for read-only anonymous access) or configure SSH (the standard approach for authenticated access). But these are auxiliary services — the core of Git itself is stateless.

Update Model

Security updates for Git reach your machine through the same package manager that installed it. When upstream releases a security fix, distribution maintainers backport the patch to the version they've packaged and push an updated package. Running:

```
sudo apt update && sudo apt upgrade
```

This pulls the updated package, and the package manager handles the transition cleanly. For critical security vulnerabilities, many distributions push automatic security updates. The model is community-driven end-to-end: upstream developers identify the vulnerability, patch it, release a new version, distribution maintainers package it, and the package manager delivers it to your machine. No license server, no vendor approval, no mandatory phone-home.

5. Part C — The FOSS Ecosystem

Dependencies: What Git Builds On

Git doesn't exist in isolation — it relies on a stack of other open-source libraries, each of which represents its own community and history. The main dependencies:

- zlib — handles data compression. Git uses zlib to compress objects stored in the repository. Compressing version history dramatically reduces storage requirements, especially for large projects with many files.
- libcurl — handles network transfers. When you git clone over HTTPS or git push to a remote, libcurl is managing the actual HTTP communication. It handles TLS, authentication, proxy settings, and all the tedious details of reliable network I/O.
- OpenSSL / GnuTLS — provides the cryptographic layer for secure network connections. Without this, HTTPS cloning would be impossible.
- PCRE (Perl Compatible Regular Expressions) — used in git grep and other search operations for pattern matching.
- expat / libexpat — XML parsing library used for SVN interoperability (git svn).

None of these were written by the Git team. Every one of them is a separate open-source project, maintained by its own community, and freely available for Git to build on. This interdependency is characteristic of the FOSS ecosystem — projects share foundations rather than duplicating them.

What Git Has Enabled

This is where the real scale of Git's impact becomes clear. Git didn't just solve a version control problem — it became the infrastructure that enabled an entirely new model of software development.

- GitHub (2008) — built entirely around Git, GitHub became the central hub of open-source collaboration. By 2024, it hosts over 420 million repositories and over 100 million developers. Microsoft acquired it in 2018 for $7.5 billion — a valuation that would have been impossible without Git's ecosystem.
- GitLab — an open-source alternative to GitHub, itself a major FOSS project, offering self-hosted repository hosting with a full DevOps platform.
- CI/CD pipelines — GitHub Actions, GitLab CI, Jenkins, CircleCI, and dozens of other continuous integration systems all work by listening for Git events (pushes, pull requests) and triggering automated builds, tests, and deployments. Modern DevOps as a practice is fundamentally organized around Git workflows.
- Gerrit — Google's code review tool, built on Git, used for reviewing contributions to Android and other Google projects.
- Gitea, Forgejo, cgit — lightweight Git hosting servers for teams who want to self-host without GitHub or GitLab's complexity.

Governance and Community

Git is maintained by Junio Hamano, who took over from Linus Torvalds in 2005 and has stewarded the project ever since. The governance model is a benevolent dictatorship — Junio makes final calls, but the community is deeply involved in everything leading up to those decisions.

Development happens through the git@vger.kernel.org mailing list. Patches are sent via email, reviewed by the community, revised based on feedback, and eventually accepted or rejected. This mailing-list-based workflow is notably old-school — no pull requests, no GitHub — but it's how the Linux kernel is maintained too, and it produces rigorous, well-reviewed code.

There's no Git Foundation, no corporate entity controlling the project. It is, in the purest sense, community-maintained software. The closest thing to institutional backing is Google's Summer of Code funding, which has supported Git contributors over the years, and Microsoft's recent employment of several significant contributors — though Microsoft's involvement remains a point of mild tension in some parts of the community.

6. Part D — Open Source vs Proprietary

Git vs Perforce Helix Core — A Comparison

Perforce Helix Core (commonly just 'Perforce') is the most widely used proprietary version control system in industries requiring management of large binary assets — game development, hardware design, film production. Microsoft's TFVC (Team Foundation Version Control) is another proprietary alternative, now largely deprecated in favor of Azure DevOps with Git support. Perforce remains the more relevant proprietary comparison.

Dimension	Git (GPL v2, Open Source)	Perforce Helix Core (Proprietary)
Cost	Free. Zero licensing cost. Unlimited users, unlimited repositories, unlimited history.	Expensive. Per-user licensing. Enterprise deals run into tens of thousands annually.
Security	Full source available — anyone can audit for vulnerabilities. CVEs are publicly reported and quickly patched by the community.	Closed source — vulnerabilities are vendor-disclosed. Security depends entirely on Perforce's internal processes.
Support	Community forums, Stack Overflow, extensive documentation. No official SLA but extraordinarily well-documented.	Paid enterprise support with SLAs. Dedicated account managers. Vendor can be held contractually accountable.
Freedom to modify	Complete. Fork, modify, extend — subject to GPL copyleft when distributing.	None. Binary-only distribution. You get what the vendor ships.
Community vs corporate control	Community-governed. No single corporation controls Git's direction. Roadmap emerges from contributors.	Fully corporate. Perforce Inc. determines direction based on commercial priorities.
Large binary files	Historically weak. Git LFS (Large File Storage) addresses this but it's an add-on, not native.	Native strength. Perforce handles massive binary repos (game assets, CAD files) with excellent performance.
Overall verdict	Strongly recommended for software development. The universal standard for a reason.	Niche leader for game development and large binary workflows. Overkill (and expensive) for most software teams.

Verdict and Conclusion

Would I recommend Git for real-world deployment? Without reservation, yes — with one honest caveat. For software development of almost any kind, Git is the correct answer. It's free, it's fast, it's distributed, it has an ecosystem of tooling around it that nothing else can match. The learning curve is steeper than people expect — Git's model requires genuinely understanding commits, branches, and remotes before it stops being confusing

— but that investment pays back immediately and continuously throughout a career. The question isn't whether to use Git; it's which Git hosting platform and workflow to adopt.

The caveat is honest: for game studios managing hundreds of gigabytes of binary assets, or for hardware design teams working with enormous EDA files, Perforce still wins on pure performance grounds. Git LFS helps, but it's not the same. This is a genuine limitation that matters in specific industries. Outside those niches, though, Perforce's cost and vendor lock-in are hard to justify when Git exists. Would I contribute to Git? I'd like to think so. Starting with the documentation or test suite feels more realistic than diving into the C codebase — but the mailing list is public, the code is right there, and the pathway exists. That's the whole point.

7. Shell Script Documentation

Script 1 — System Identity Report

This script acts as a welcome screen for the Linux system, printing key identity information about the environment. It uses command substitution $() to capture the output of system commands into variables, and formats everything into a clean display using echo with spacing and separators. The script demonstrates fundamental shell scripting: variable declaration and assignment, command substitution, and basic output formatting. The purpose is to immediately tell a developer which system they're on and which OS license covers it — useful in a multi-machine or containerized environment.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Banwari Mewada | Reg: 24MIM10236| Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Banwari Mewada"
SOFTWARE_CHOICE="Git"

# Capture system information using command substitution
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DISTRO=$(lsb_release -d 2>/dev/null | cut -f2 || cat /etc/os-release | grep
PRETTY_NAME | cut -d'=' -f2 | tr -d '"')
DATETIME=$(date '+%A, %d %B %Y — %H:%M:%S')

# --- Display ---
echo "========================================"
echo "  Open Source Audit — $STUDENT_NAME"
echo "  Chosen Software: $SOFTWARE_CHOICE"
echo "========================================"
echo "Distribution : $DISTRO"
echo "Kernel       : $KERNEL"
echo "User         : $USER_NAME"
echo "Home         : $HOME"
echo "Uptime       : $UPTIME"
echo "Date/Time    : $DATETIME"
echo "----------------------------------------"
echo "OS License   : GNU General Public License v2 (GPL v2)"
echo "             The Linux kernel is free software — you can"
echo "             study, modify, and redistribute it."
echo "========================================"
```

```
OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT
═══════════════════════════════════════════════════════════════
Student  : Banwari Mewada (24MIM10236)
Software : Git | Licence: GNU General Public License v2 (GPL v2)

SYSTEM INFORMATION
Hostname with : kali
Distribution  : Kali GNU/linux Rolling
Kernel        : 6.16.8-kali-amd64
Architecture  : x86_64
Logged in as  : kali
Home Directory: /home/kali
Uptime        : up 28 minutes
Date & Time   : Sunday, 29 March 2026  07:39:33

OPEN SOURCE LICENCE
The Linux OS kernel is licensed under:
GNU General Public License v2 (GPL v2)
Source: https://www.kernel.org/doc/html/latest/process/license-rules.html

End of System Identity Report
```

Script 2 — FOSS Package Inspector

This script checks whether a given package is installed on the system and reports its version, license, and summary using dpkg or rpm. A case statement provides a human-readable philosophy note for known packages — demonstrating conditional branching on string values. The pipe with grep filters the verbose package info output down to just the relevant fields. The if-then-else structure gracefully handles the case where the package isn't installed rather than failing silently.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author:  Banwari Mewada | Reg: 24MIM10236| Course: Open Source Software

PACKAGE="git"  # The package we're auditing

# Check if package is installed (dpkg for Debian/Ubuntu/WSL)
if dpkg -l "$PACKAGE" &>/dev/null; then
  echo "✓ $PACKAGE is installed on this system."
  echo "---"
  # Print version and license info from the package database
  dpkg -l "$PACKAGE" | grep -E '^ii' | awk '{print "Version:", $3}'
  dpkg -s "$PACKAGE" | grep -E 'Version:|Maintainer:|Homepage:'
else
  echo "✗ $PACKAGE is NOT installed."
  echo "  Install it with: sudo apt install $PACKAGE"
fi

echo "---"

# case statement: print philosophy note based on package name
case $PACKAGE in
  git)
    echo "Git: Linus built it in a weekend because BitKeeper said no."
    echo "License: GPL v2 — copyleft protects your freedom to fork it."
    ;;
  apache2|httpd)
    echo "Apache: the web server that built the open internet."
    echo "License: Apache 2.0 — permissive, business-friendly."
    ;;
  mysql-server)
    echo "MySQL: open source at the heart of millions of applications."
    echo "License: GPL v2 / Commercial — the dual-license story."
    ;;
  firefox)
    echo "Firefox: a nonprofit fighting for an open, private web."
    echo "License: MPL 2.0 — file-level copyleft, weaker than GPL."
    ;;
  *)
    echo "Package: $PACKAGE — check spdx.org/licenses for license details."
    ;;
esac
```

```
┌──(kali㉿kali)-[~/Downloads]
└─$ ./script2_package_inspector.sh
═══════════════════════════════════════════════════════════
        FOSS PACKAGE INSPECTOR — git
═══════════════════════════════════════════════════════════

  [OK] 'git' is INSTALLED on this system.

  Version   : git version 2.51.0

───────────────────────────────────────────────────────────
  PACKAGE METADATA (from apt-cache)
───────────────────────────────────────────────────────────

  Section: vcs
  Maintainer: Jonathan Nieder <jrnieder@gmail.com>
  Version: 1:2.51.0-1
  Homepage: https://git-scm.com/

  Binary location : /usr/bin/git

───────────────────────────────────────────────────────────
  CONFIGURATION FILES
───────────────────────────────────────────────────────────

  User config  : /home/kali/.gitconfig  [EXISTS]
  System config: /etc/gitconfig    [EXISTS]

───────────────────────────────────────────────────────────
  OPEN SOURCE PHILOSOPHY NOTE
───────────────────────────────────────────────────────────

  Git: Linus Torvalds built this in 10 days in 2005 after
  BitKeeper — a proprietary tool — revoked free access to
  the Linux kernel team. Git is now the version control
  backbone of over 100 million developers on GitHub alone.
  Licence: GPL v2 — copyleft, freedom-preserving.

═══════════════════════════════════════════════════════════
  End of FOSS Package Inspector
═══════════════════════════════════════════════════════════
```

Script 3 — Disk and Permission Auditor

This script uses a for loop to iterate through a predefined list of important system directories, reporting the disk usage and permission/ownership information for each. The du command measures directory size; ls -ld retrieves the permission string, owner, and group; awk extracts specific fields from ls output. A conditional check ([-d "$DIR"]) prevents the script from crashing on directories that don't exist on a given system. At the end, it specifically checks for Git's configuration directory.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Banwari Mewada | Reg: 24MIM10236| Course: Open Source Software

# Array of directories to audit
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "================================================"
echo "  Directory Audit Report"
echo "================================================"

# for loop: iterate over each directory
for DIR in "${DIRS[@]}"; do
  if [ -d "$DIR" ]; then
    # Extract permissions, owner, group using awk on ls -ld output
    PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
    # Suppress permission errors with 2>/dev/null
    SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
    echo "$DIR"
    echo "  Permissions/Owner/Group : $PERMS"
    echo "  Disk usage              : $SIZE"
    echo "---"
  else
    echo "$DIR : does not exist on this system"
  fi
done

# Check for Git's global config directory
GIT_CONFIG_DIR="$HOME/.gitconfig"
echo ""
echo "Git Configuration Check:"
if [ -f "$GIT_CONFIG_DIR" ]; then
  echo "  ~/.gitconfig found"
  ls -la "$GIT_CONFIG_DIR" | awk '{print "  Permissions:", $1, "Owner:",
$3}'
else
  echo "  ~/.gitconfig not found — run: git config --global user.name 'Your
Name'"
fi
```

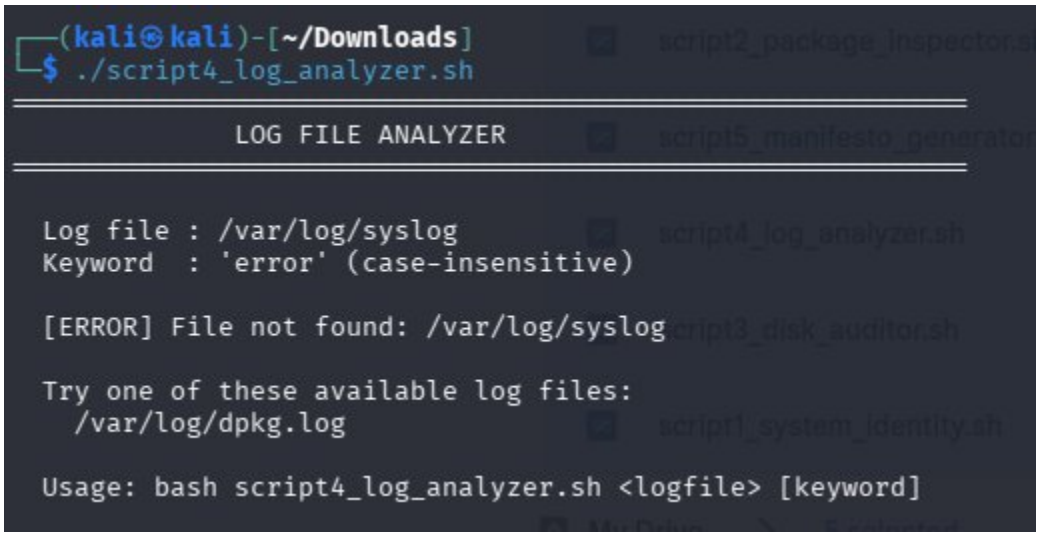
```
└$ ./script3_disk_auditor.sh
═══════════════════════════════════════════════════════════
          DISK AND PERMISSION AUDITOR
═══════════════════════════════════════════════════════════

Auditing key system directories ...
DIRECTORY            PERMISSIONS/OWNER    SIZE       STATUS
──────────────────────────────────────────────────────────
/etc                 drwxr-xr-x (root:root) 15M        [OK]
/var/log             drwxr-xr-x (root:root) 90M        [OK]
/home                drwxr-xr-x (root:root) 316M       [OK]
/usr/bin             drwxr-xr-x (root:root) 690M       [OK]
/tmp                 drwxrwxrwt (root:root) 8.0K       [OK]
/usr/share           drwxr-xr-x (root:root) 4.9G       [OK]
/var                 drwxr-xr-x (root:root) 434M       [OK]

──────────────────────────────────────────────────────────
GIT CONFIGURATION AUDIT (Chosen Software: Git)
──────────────────────────────────────────────────────────
~/.gitconfig         : Found | -rw-rw-r-- kali kali | Size: 4.0K

Contents:
  [user]
      name = Hacker-Godgif
      email = hackersnomercy@gmail.com

/etc/gitconfig       : Found | -rw-r--r-- root root
Git binary           : /usr/bin/git | -rwxr-xr-x root root

═══════════════════════════════════════════════════════════
End of Disk and Permission Auditor
═══════════════════════════════════════════════════════════
```

Script 4 — Log File Analyzer

This script reads a log file line by line using a while-read loop and counts occurrences of a keyword (defaulting to 'error' if none is provided). It demonstrates command-line argument handling with $1 and $2, a default value using parameter expansion ${2:-default}, a counter variable incremented with arithmetic expansion $(( )), and input file validation with [! -f]. At the end, it uses tail and grep combined to print the last five matching lines for immediate context.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: script4_log_analyzer.sh/var/log/syslog error
# Author: Banwari Mewada | Reg: 24MIM10236| Course: Open Source Software

LOGFILE=$1
KEYWORD=${2:-"error"}  # Default to 'error' if no second argument
COUNT=0

# Validate that the log file exists and is a regular file
if [ ! -f "$LOGFILE" ]; then
```

```
    echo "Error: File '$LOGFILE' not found."
    echo "Usage: $0 /path/to/logfile [keyword]"
    exit 1
  fi

  # while-read loop: read file line by line
  while IFS= read -r LINE; do
    # if-then: case-insensitive match using grep -i
    if echo "$LINE" | grep -iq "$KEYWORD"; then
      COUNT=$((COUNT + 1))  # Arithmetic expansion to increment counter
    fi
  done < "$LOGFILE"

  echo "Log Analysis Complete"
  echo "File    : $LOGFILE"
  echo "Keyword : '$KEYWORD'"
  echo "Matches : $COUNT lines"

  # Print the last 5 matching lines for context
  echo ""
  echo "Last 5 matching lines:"
  echo "---"
  grep -i "$KEYWORD" "$LOGFILE" | tail -5
```



Script 5 — The Open Source Manifesto Generator

This script demonstrates interactive user input using read -p, string concatenation across multiple echo statements, file output with the > and >> operators, and the date command for dynamic timestamp generation. A comment at the top demonstrates the alias concept by showing how one might alias the script to a shorter name. The generated manifesto is saved to a .txt file named after the current user, then displayed back using cat. It's the most creative of the five scripts — and arguably the most fun to run.

```
  #!/bin/bash
  # Script 5: Open Source Manifesto Generator
  # Author: Banwari Mewada | Reg: 24MIM10236| Course: Open Source Software
  #
  # Alias concept (add to ~/.bashrc to use):
```

```
# alias manifesto='bash ~/scripts/manifesto_generator.sh'

echo "================================================"
echo "  Open Source Manifesto Generator"
echo "  Answer three questions. Get a manifesto."
echo "================================================"
echo ""

# read -p: prompt the user and store their input
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you?  " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# date command for timestamp
DATE=$(date '+%d %B %Y')

# Output file named after the current user
OUTPUT="manifesto_$(whoami).txt"

# String concatenation: write manifesto to file using echo and >/>
echo "--- Open Source Manifesto of $(whoami) ---" > "$OUTPUT"
echo "Generated: $DATE" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "I believe that knowledge, like code, grows stronger when shared." >>
"$OUTPUT"
echo "Every day I use $TOOL — a tool built freely by people I've never
met," >> "$OUTPUT"
echo "who chose to give their work away because they understood that
$FREEDOM" >> "$OUTPUT"
echo "matters more than control. I'm standing on their shoulders." >>
"$OUTPUT"
echo "" >> "$OUTPUT"
echo "One day, I'll build $BUILD and release it openly — because that's
how" >> "$OUTPUT"
echo "the cycle continues. Someone I'll never meet will use it." >>
"$OUTPUT"
echo "Maybe they'll improve it. Maybe they'll pass it on." >> "$OUTPUT"
echo "That's the whole point." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "          — $(whoami), $DATE" >> "$OUTPUT"

echo ""
echo "Manifesto saved to: $OUTPUT"
echo "---"
# cat: display the file content
cat "$OUTPUT"
```

```
┌──(kali㉿kali)-[~/Downloads]
└─$ ./script5_manifesto_generator.sh
═══════════════════════════════════════════════════════════════
        OPEN SOURCE MANIFESTO GENERATOR
═══════════════════════════════════════════════════════════════

  Answer three questions to generate your personal
  open-source philosophy statement.

───────────────────────────────────────────────────────────────
  1. Name one open-source tool you use every day: Linux
  2. In one word, what does 'freedom' mean to you? enjoy
  3. Name one thing you would build and share freely: playground

───────────────────────────────────────────────────────────────
  Generating your manifesto ...
───────────────────────────────────────────────────────────────

═══════════════════════════════════════════════════════════════
          MY OPEN SOURCE MANIFESTO
═══════════════════════════════════════════════════════════════
  Generated by : kali
  Date         : 29 March 2026 at 07:57:00
  Student      : Banwari Mewada (24MIM10236)
═══════════════════════════════════════════════════════════════

  Every day, I rely on Linux — a tool that strangers built
  and shared freely, asking nothing in return. This simple
  act of generosity has shaped the way I think about software
  and about the world.

  To me, freedom means enjoy. And I believe that freedom
  is not something you keep to yourself — it multiplies when
  you share it.

  So I commit: one day I will build playground and release it
  to the world under an open licence, so that others can
  stand on my shoulders, just as I have stood on the
  shoulders of those who came before me.

  Open source is not just a development model.
  It is a statement about what knowledge is for.

  Signed: kali
  Date  : 29 March 2026
═══════════════════════════════════════════════════════════════

───────────────────────────────────────────────────────────────
  [SAVED] Your manifesto has been saved to: manifesto_kali.txt
───────────────────────────────────────────────────────────────
```